



MÓDULO 11: Modularización, Creación y Empaquetado de Librerías Reutilizables en Python

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 11 - Python Packaging, Standard Project Layout (`src/`), `pyproject.toml` , `pip install -e .` & pyOpenSci Best Practices

1. Objetivos de Aprendizaje

- **Estructura Estándar de Proyectos Python:** Comprender el diseño moderno de paquetes en Python utilizando la arquitectura con directorio de código fuente `src/` y archivos de inicialización `__init__.py`.
- **Configuración con `pyproject.toml`:** Definir los metadatos del proyecto (nombre, versión, autor, dependencias, licencias) mediante el estándar PEP 517 / PEP 621.
- **Herramientas de Generación y Empaquetado (`copier` , `hatch`):** Conocer la utilización de plantillas estandarizadas (*templates*) del proyecto **pyOpenSci** para la reproducibilidad de la Ciencia Abierta.
- **Modo Editable (`pip install -e .`):** Instalar paquetes propios en modo desarrollo para que los cambios en el código fuente se reflejen instantáneamente sin reinstalar.

- **Creación de Módulos Reutilizables:** Diseñar funciones personalizadas y clases de dominio científico (ej. clase `Persona` con control de institutos y servicios del CSIC).

2. ¿Por qué Empaquetar nuestro Código en Ciencia Abierta?

A medida que nuestros proyectos de investigación crecen, pasar de scripts `.py` sueltos a **paquetes de Python estructurados** es un paso fundamental para garantizar la **reproducibilidad**, la reutilización de código entre miembros del grupo y la distribución abierta.

Beneficios Principales:

1. **Reutilización sin Copiar-Pegar:** Puedes importar tu código desde cualquier lugar de tu equipo o en otros cuadernos sin tener que copiar archivos `.py`.
2. **Distribución Sencilla:** Permite que otros investigadores del CSIC o de la comunidad internacional instalen tu librería con un simple `pip install`.
3. **Organización Estandarizada:** Separa claramente el código fuente (`src/`), las pruebas unitarias (`tests/`), la documentación (`docs/`) y la configuración del proyecto (`pyproject.toml`).

3. Paso 1: Generación de la Estructura de Proyecto con Plantillas

Para evitar crear manualmente decenas de archivos de configuración, utilizamos herramientas de generación de proyectos como **copier** recomendadas por la comunidad **pyOpenSci**.

Herramientas Clave:

- **Copier** : Generador de plantillas que automatiza la creación de la estructura del paquete.
- **Hatch** : Gestor moderno de empaquetado, dependencias y entornos virtuales en Python.

Instalación de las Herramientas de Empaquetado (`copier` y `hatch`)

Para poder generar la estructura estandarizada de proyectos y gestionar el empaquetado de nuestras librerías, necesitaremos instalar las siguientes herramientas en nuestro entorno virtual:

- **copier** : Motor de plantillas para generar la estructura de archivos de la librería.
- **hatch** : Administrador moderno de proyectos y empaquetado Python.

Comando de Instalación:

Nuestro archivo de requerimientos del curso ya las incluye (`requirements.txt`), pero si las quieres instalar desde la línea de comandos simplemente ejecuta:

```
pip install copier hatch
```

Verificación e Instalación de Herramientas en la Terminal:

En tu terminal con el entorno virtual activo (`source venv/bin/activate`), verifica la instalación de las herramientas:

```
copier --version
hatch --version
```

Para generar la estructura completa de un nuevo paquete según las recomendaciones científicas de **pyOpenSci**, vamos al terminal y **CON EL ENTORNO VIRTUAL ACTIVADO** ejecutamos:

```
copier copy gh:pyopensci/pyos-package-template
./PYTHON_CIENCIA_ABIERTA_2
```

1. Elige la configuracion **MINIMAL**
 2. PROJECT NAME: **PYTHON_CIENCIA_ABIERTA_2**
 3. Descripción: Librería de ejemplo para el empaquetado de código en el Curso Python para la Ciencia Abierta . Pulsa ESC y ENTER
 4. Author: **TU NOMBRE**
 5. Email: **TU EMAIL**
 6. Github username: **Tu nombre de usuario en GitHub**
-

Estructura Estandarizada Generada:

```
PYTHON_CIENCIA_ABIERTA_2/
├── .gitignore
├── CHANGELOG.md
├── LICENSE
├── README.md
├── pyproject.toml          <-- Corazón de la configuración del
paquete
├── src/                   <-- Código fuente del paquete
│   └── __init__.py        <-- Indica a Python que es un
```

```

paquete
├── example.py          <-- Módulos con funciones y clases
├── tests/
│   └── test_example.py <-- Pruebas unitarias automatizadas

```

4. Paso 2: El Corazón de la Configuración: `pyproject.toml`

El archivo `pyproject.toml` es el estándar moderno de la comunidad Python (PEP 517 / PEP 621) para definir cómo se construye, instala y publica un paquete.

Ejemplo de Configuración Estándar:

```

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "test-pyopensci"
version = "0.1.0"
description = "Paquete Python de ejemplo para Ciencia Abierta en el CSIC"
readme = "README.md"
requires-python = ">=3.10"
authors = [
    { name = "Gustavo Liñán Cembrano", email = "gustavo.linan@csic.es" }
]
dependencies = [
    "pandas>=2.0.0",
    "requests>=2.28.0",
    "beautifulsoup4>=4.11.0"
]

[project.urls]
Homepage = "https://github.com/guslicem/PYTHON_PARA_CIENCIA_ABIERTA_2"

```

4.a: Edita el archivo `pyproject.toml` y añade `pytest` a las dependencias

1. Abre el archivo `pyproject.toml` ubicado en la raíz de tu paquete (`PYTHON_CIENCIA_ABIERTA_2/pyproject.toml`).
2. Localiza la sección `[project]` y busca el parámetro `dependencies`.
3. Añade `"pytest"` a la lista de dependencias:

5. Paso 3: Instalación del Paquete en Modo Editable (`pip install -e .`)

Durante la fase de investigación y desarrollo, es muy conveniente instalar el paquete en **modo editable** (`-e` o `--editable`).

¿Qué hace el flag `-e`?

En lugar de copiar los archivos del paquete al directorio de librerías de Python (`site-packages`), la instalación en modo editable crea una **referencia simbólica o enlace** hacia tu carpeta de código en desarrollo.

Esto significa que **cualquier cambio que guardes en los archivos `.py` de la carpeta `src/` se reflejará inmediatamente en tus cuadernos sin necesidad de volver a ejecutar `pip install`.**

```
# Navegamos a la carpeta del paquete
cd NOTEBOOKS/PYTHON_CIENCIA_ABIERTA_2
hatch shell
```

Observa como hatch ha creado un entorno virtual propio para tu paquete lo que te va a permitir trabajar con él sin provocar interacciones no-deseadas con otros paquetes en nuestro sistema.

Instalamos el paquete localmente en modo editable

```
pip install -e .
```

6. Desarrollo de Módulos: Funciones Creadas y Clases de Dominio Científico

A continuación implementamos en el código fuente de nuestro paquete dos componentes fundamentales:

1. Una función matemática recursiva (`my_factorial`).
2. Una clase de dominio para gestionar investigadores y personal de apoyo del CSIC (`Persona`).

```
In [11]: """
Módulo usuario_lib.py
=====
Librería de funciones matemáticas y utilidades para el paquete de Ciencia Abierta
Autor: Curso Python para Ciencia Abierta (CSIC)
```

```

"""
def my_factorial(x: int) -> int:
    """
    Calcula el factorial de un número entero no negativo utilizando recursión
    Parameters
    -----
    x : int
        Número entero no negativo.
    Returns
    -----
    int
        El factorial de x.
    Raises
    -----
    ValueError
        Si x es un número negativo.
    """
    if x < 0:
        raise ValueError("x debe ser un entero no negativo")
    if x == 0:
        return 1
    if x == 1:
        return 1
    return x * my_factorial(x - 1)
# --- Bloque de Ejecución Directa para Testeo ---
if __name__ == "__main__":
    print("=" * 60)
    print("🔧 Probando el módulo usuario_lib.py localmente:")
    print("=" * 60)
    print("\nCálculo de factoriales de números impares del 1 al 9:")
    for num in range(1, 10, 2):
        print(f" - Factorial de {num}!: {my_factorial(num)}")
    print("\n✅ Pruebas locales completadas exitosamente.")

```

```

=====
🔧 Probando el módulo usuario_lib.py localmente:
=====

```

Cálculo de factoriales de números impares del 1 al 9:

- Factorial de 1!: 1
- Factorial de 3!: 6
- Factorial de 5!: 120
- Factorial de 7!: 5040
- Factorial de 9!: 362880

✅ Pruebas locales completadas exitosamente.

```

In [12]: """
Módulo usuario_class.py
=====
Definición de la clase Persona para la gestión de adscripciones del personal

Autor: Curso Python para Ciencia Abierta (CSIC)
"""

```

```

class Persona:
    """
    Clase que representa a un miembro del personal del CSIC.
    """
    # Listas estáticas de validación de institutos y servicios del CSIC
    INSTITUTOS_CSIC = [
        "IMSE",          # Instituto de Microelectrónica de Sevilla (IMSE-CNM)
        "IMB",           # Instituto de Microelectrónica de Barcelona
        "EBD",           # Estación Biológica de Doñana
        "CSIC-URICI",    # Unidad de Recursos de Información Científica para
        "CSIC-SGAI"      # Secretaría General Adjunta de Informática
    ]

    SERVICIOS_CSIC = [
        "Biblioteca",
        "Informática",
        "Administración",
        "Comunicación",
        "Heladería",
        "Investigación"
    ]

    def __init__(self, nombre: str, edad: int, instituto: str = None, servicio: str = None):
        self.nombre = nombre
        self.edad = edad
        self._instituto = None
        self._servicio = None

        # Asignar validando a través de los setters de propiedades
        if instituto:
            self.instituto = instituto
        if servicio:
            self.servicio = servicio

    # --- Propiedad INSTITUTO (Getter y Setter con validación) ---
    @property
    def instituto(self):
        return self._instituto

    @instituto.setter
    def instituto(self, valor):
        if valor is not None and valor not in self.INSTITUTOS_CSIC:
            raise ValueError(f"Instituto no válido: '{valor}'. Opciones válidas: {self.INSTITUTOS_CSIC}")
        self._instituto = valor

    # --- Propiedad SERVICIO (Getter y Setter con validación) ---
    @property
    def servicio(self):
        return self._servicio

    @servicio.setter
    def servicio(self, valor):
        if valor is not None and valor not in self.SERVICIOS_CSIC:
            raise ValueError(f"Servicio no válido: '{valor}'. Opciones válidas: {self.SERVICIOS_CSIC}")
        self._servicio = valor

```

```

# --- Métodos de Comportamiento ---
def saludar(self) -> str:
    """Devuelve un saludo formal indicando la adscripción institucional.
    if self.instituto:
        return f"Hola, me llamo {self.nombre}, tengo {self.edad} años y
    elif self.servicio:
        return f"Hola, me llamo {self.nombre}, tengo {self.edad} años y
    else:
        return f"Hola, me llamo {self.nombre}, tengo {self.edad} años."

def crecer(self) -> int:
    """Incrementa la edad de la persona en un año (cumpleaños)."""
    self.edad += 1
    return self.edad

# --- Bloque de Ejecución Directa para Testeo ---
if __name__ == "__main__":
    print("=" * 60)
    print("🔧 Probando el módulo usuario_class.py localmente:")
    print("=" * 60)

    # Instanciar miembro del CSIC
    persona_test = Persona(nombre="Gustavo Liñán", edad=53, instituto="IMSE")
    print(f"\n1. Saludo inicial: {persona_test.saludar()}")

    # Probar método crecer
    persona_test.crecer()
    print(f"2. Cumpleaños: {persona_test.saludar()}")

    # Probar cambio de adscripción a un servicio
    persona_test.instituto = None
    persona_test.servicio = "Heladeria"
    print(f"3. Cambio de servicio: {persona_test.saludar()}")

    print("\n✅ Pruebas locales de la clase Persona completadas con éxito.")

```

```

=====
🔧 Probando el módulo usuario_class.py localmente:
=====

```

1. Saludo inicial: Hola, me llamo Gustavo Liñán, tengo 53 años y pertenezco al instituto IMSE.
2. Cumpleaños: Hola, me llamo Gustavo Liñán, tengo 54 años y pertenezco al i nstituto IMSE.
3. Cambio de servicio: Hola, me llamo Gustavo Liñán, tengo 54 años y trabajo en el servicio de Heladeria.

✅ Pruebas locales de la clase Persona completadas con éxito.

Ya que hemos probado las funciones, navega a la carpeta src del directorio de tu proyecto de software y crea un archivo con, por ejemplo, tu nombre + _lib.py y otro con tu nombre y _class.py. En mi caso gus_lib.py gus_class.py. Pega en el primero la funcion factorial y en el segundo la clase Persona que hemos creado.

7. La Importancia de las Pruebas Unitarias (*Testing*) en Ciencia Abierta

En el desarrollo de software científico y la investigación abierta, escribir **pruebas unitarias** (*unit tests*) es la garantía fundamental de que **nuestros cálculos, algoritmos y flujos de datos son correctos, estables y reproducibles**.

🎯 ¿Por qué es vital definir tests para las funciones que creamos?

1. **Garantía de Correctitud Científica:** Un fallo no detectado en una función matemática o en una rutina de procesamiento puede alterar los resultados de un experimento. Las pruebas verifican de forma automatizada que las funciones devuelven exactamente los valores esperados ante casos límite y de uso normal.
2. **Prevención de Regresiones (Evitar romper lo que funcionaba):** A medida que el paquete crece o refactorizamos código, ejecutar las pruebas nos avisa de inmediato si un nuevo cambio ha roto una funcionalidad que previamente funcionaba bien.
3. **Confianza en el Trabajo Colaborativo:** En proyectos con varios investigadores o grupos de trabajo del CSIC, los tests permiten que cualquier miembro añada mejoras sin miedo a desestabilizar el trabajo de sus compañeros.
4. **Documentación Viva y Verificable:** Un archivo de prueba (`test_*.py`) es el mejor ejemplo práctico de uso: muestra de manera exacta e inequívoca qué entradas recibe una función, qué salidas debe generar y qué excepciones debe lanzar.
5. **Integración Continua (CI/CD con GitHub Actions):** En paquetes modernos (como el generado por **pyOpenSci**), las pruebas se ejecutan automáticamente en servidores remotos cada vez que enviamos cambios a GitHub, garantizando que el paquete funcione en diferentes sistemas operativos y versiones de Python.

7.a Creamos un archivo de pruebas para la funcion factorial

En la estructura estándar recomendada por **pyOpenSci**, los archivos de pruebas se guardan dentro de la carpeta `tests/` (o `tests/unit/`). Para que el motor de pruebas (**pytest**) reconozca automáticamente las funciones de test, **el nombre del archivo debe comenzar siempre con el prefijo `test_`**.

7.a.1: Navega a

`./NOTEBOOKS/PYTHON_CIENCIA_ABIERTA_2/tests/unit/` y borra el archivo por defecto `test_example.py`

7.a.2: Crea el archivo:

`test_my_factorial.py`

7.a.3: Copia este contenido en el archivo:

```
"""
Módulo de pruebas unitarias para la función my_factorial.
"""

import pytest
from python_ciencia_abierta_2.gus_lib import my_factorial

def test_my_factorial_casos_base():
    msg= """Verifica los casos base de la función factorial: 0! = 1
    y 1! = 1."""
    print(msg)
    assert my_factorial(0) == 1
    assert my_factorial(1) == 1

def test_my_factorial_valores_positivos():
    msg= """Verifica el cálculo correcto para varios enteros
    positivos conocidos."""
    print(msg)
    assert my_factorial(3) == 6
    assert my_factorial(5) == 120
    assert my_factorial(7) == 5040

def test_my_factorial_error_negativos():
    msg= """Verifica que la función lanza la excepción ValueError al
    recibir enteros negativos."""
    print(msg)
    with pytest.raises(ValueError, match="x debe ser un entero no
    negativo"):
        my_factorial(-1)
    with pytest.raises(ValueError):
        my_factorial(-5)
```

7.a.4: Abre un terminal, navega a la carpeta del proyecto

`cd PYTHON_CIENCIA_ABIERTA_2`

Ejecuta el shell de hatch

`hatch shell`

Instala el proyecto en modo edicion en el entorno virtual

`pip install -e .`

Ejecuta el paquete de tests con el comando `pytest` en el terminal

`pytest`

NOTA: El Gestor de Contexto `with` `pytest.raises(...)` en las pruebas

En el desarrollo de software de calidad, **no solo debemos probar que las funciones funcionen bien cuando se les proporcionan datos correctos** (los llamados "*casos felices*"); también es imprescindible verificar que el programa **reaccione correctamente e interrumpa la ejecución controladamente** cuando recibe datos inválidos o erróneos.

? ¿Qué hace la instrucción `with` con `pytest.raises`?

La palabra clave `with` abre lo que en Python se denomina un **Gestor de Contexto** (*Context Manager*). Al combinarlo con `pytest.raises(...)`:

1. **Captura el Error:** Le dice a `pytest`: "*Ejecuta el bloque de código indentado a continuación y espera que se produzca una excepción del tipo especificado*".
 2. **Pasa el test si se lanza el error:** Si la línea dentro del bloque efectivamente lanza la excepción esperada (por ejemplo, `ValueError`), **el test se considera EXITOSO (PASSED)**.
 3. **Falla el test si NO hay error:** Si el código dentro del bloque se ejecuta limpiamente sin lanzar ningún error, **el test FALLA (FAILED)** porque la validación de seguridad de nuestro código no se activó.
-

7.b: Pruebas Unitarias para la Clase Persona (`test_persona.py`)

A continuación creamos un nuevo archivo de pruebas unitarias para validar de forma integral el comportamiento de la clase `Persona`, incluyendo la instanciación de investigadores, la ejecución de sus métodos (`saludar()`, `crecer()`), la actualización de propiedades con validación y la adición de nuevos centros del CSIC.

7.b.1: Navega a

`./NOTEBOOKS/PYTHON_CIENCIA_ABIERTA_2/test`
Y crea el archivo: `test_persona.py`

 Código para `test_persona.py`:

.....

Módulo de pruebas unitarias para la clase `Persona` (adscripciones

```

del CSIC).
"""

import pytest
from python_ciencia_abierta_2.gus_class import Persona
def test_persona_creacion_y_saludo():
    """Verifica la instanciación correcta de Persona y el saludo de
    bienvenida."""
    msg = "Verificando creación de investigador y método
    saludar()..."
    print(msg)
    p = Persona(nombre="Gustavo Liñán", edad=53, instituto="IMSE")
    assert p.nombre == "Gustavo Liñán"
    assert p.edad == 53
    assert p.instituto == "IMSE"
    assert "pertenezco al instituto IMSE" in p.saludar()
def test_persona_metodo_crecer():
    """Verifica el incremento de edad al cumplir años."""
    msg = "Verificando método crecer()..."
    print(msg)
    p = Persona(nombre="Gustavo Liñán", edad=53, instituto="IMSE")
    nueva_edad = p.crecer()
    assert nueva_edad == 54
    assert p.edad == 54
def test_persona_cambio_servicio():
    """Verifica el cambio dinámico de adscripción desde un
    instituto a un servicio."""
    msg = "Verificando cambio de adscripción al servicio de
    Heladería..."
    print(msg)
    p = Persona(nombre="Gustavo Liñán", edad=53, instituto="IMSE")
    p.instituto = None
    p.servicio = "Heladeria"
    assert p.instituto is None
    assert p.servicio == "Heladeria"
    assert "trabajo en el servicio de Heladeria" in p.saludar()
def test_persona_anadir_nuevo_instituto():
    """Verifica la incorporación de un nuevo centro CSIC a la lista
    estática INSTITUTOS_CSIC."""
    msg = "Verificando adición de un nuevo centro (IIBB) a la lista
    estática..."
    print(msg)
    if "IIBB" not in Persona.INSTITUTOS_CSIC:
        Persona.INSTITUTOS_CSIC.append("IIBB")

    p = Persona(nombre="Dra. María Carmen", edad=41,
    instituto="IIBB")
    assert p.instituto == "IIBB"
    assert "pertenezco al instituto IIBB" in p.saludar()
def test_persona_error_instituto_invalido():
    """Verifica que se lanza ValueError al intentar asignar un
    instituto no reconocido."""
    msg = "Verificando captura de excepciones (ValueError) con

```

```

institutos inválidos..."
    print(msg)
    with pytest.raises(ValueError, match="Instituto no válido"):
        Persona(nombre="Investigador X", edad=30,
instituto="INSTITUTO_INEXISTENTE")

```

Ejecuta el paquete de tests con el comando `pytest` en el terminal

`pytest`

El resultado en tu terminal tendrá una pinta así:

```

(python-ciencia-abierta-2) (base) linan@MacBook-Air-de-Gustavo
PYTHON_CIENCIA_ABIERTA_2 % py test
===== test session starts
===== platform darwin -- Python 3.12.7,
pytest-9.1.1, pluggy-1.6.0 rootdir:
/Users/linan/Desktop/RESEARCH/FORMACION/PYTHON_PARA_CIENCIA_ABIERTA_2/NOTEE
configfile: pyproject.toml testpaths: tests collected 8 items

tests/unit/test_my_factorial.py ... [ 37%] tests/unit/test_persona.py ..... [100%]

===== 8 passed in 0.01s
=====

```

El porcentaje solo indica el progreso del proceso de test, no que se hayan o no pasado con éxito los mismos. A medida que va terminando de ejecutar cada archivo de prueba, muestra qué porcentaje del trabajo total ha completado:

Al terminar `test_my_factorial.py`:

Ha ejecutado 3 de las 8 pruebas.

$3/8 = 37\%$

Al terminar `test_persona.py`:

Ha ejecutado las 5 pruebas restantes (llegando a 8 de 8) (100%)

Ejercicio Práctico: Provocar y Diagnosticar un Test Fallido en Pytest

Una de las mayores ventajas de las pruebas unitarias es su capacidad para **detectar inmediatamente cuándo un cambio en el código introduce un error** (lo que en ingeniería de software se conoce como una *regresión*).

En este ejercicio vamos a simular la introducción intencionada de un error lógico en nuestra librería para observar cómo `pytest` aísla el fallo y nos señala con precisión

quirúrgica la línea afectada.

Paso 1: Introducir un error lógico en `usuario_lib.py`

Abre el archivo de tu librería y modifica el caso base de la función `my_factorial` para que erróneamente devuelva `0` en lugar de `1` cuando `x == 0`:

```
#  Modificación intencionadamente errónea para la práctica:
def my_factorial(x: int) -> int:
    if x < 0:
        raise ValueError("x debe ser un entero no negativo")
    if x == 0:
        return 0 #  ERROR LÓGICO INTENCIONADO: Matemáticamente
0! = 1
    if x == 1:
        return 1
    return x * my_factorial(x - 1)
```

Paso 2: Ejecutar las Pruebas desde la Terminal

Vuelve a ejecutar pytest en la terminal (con hatch shell activo):

```
pytest tests/unit/test_my_factorial.py
```

Paso 3: Diagnosticando el fallo

Observa cómo Pytest marca la prueba en rojo con FAILED y genera una comparativa (diff) clara entre el valor obtenido (0) y el valor esperado (1):

```
text ===== FAILURES
=====
test_my_factorial_casos_base _____ def
test_my_factorial_casos_base(): """Verifica los casos base de la función factorial: 0! = 1
y 1! = 1."""

    assert my_factorial(0) == 1
```

E assert 0 == 1 E + where 0 = my_factorial(0) tests/unit/test_my_factorial.py:8:

AssertionError ===== short test summary info

===== FAILED

tests/unit/test_my_factorial.py::test_my_factorial_casos_base - assert 0 == 1

===== 1 failed, 2 passed in 0.02s

=====

Paso 4: Restauramos

Restaura el archivo `usuario_lib.py` corrigiendo para que el factorial de 0 sea 1 y vuelve a lanzar `pytest` para verificar que la suite vuelve al estado 100% PASSED (Verde) 🟢.

Vuelve a ejecutar `pytest` en la terminal (con `hatch shell` activo):

```
pytest tests/unit/test_my_factorial.py
```



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:



Cuestionario Módulo 11: Modularización y Empaquetado de Librerías
